

# MOCK INTERVIEW QUESTIONNAIRE

## Java Full Stack Development

*Cognizant Digital Nurture 5.0 — Deep Skilling Handbook (Java FSE / React Track)*

Prepared for Trainer Use — Final-Year CSE Batch

Department of Computer Engineering and Applications, IET

### Instructions for the Trainer

This questionnaire spans 12 topic areas mapped to the DN 5.0 Java Full Stack curriculum. It is designed for a 25 to 30-minute mock interview.

- Difficulty levels: Easy (fundamentals), Medium (applied understanding), Hard (design/trade-off reasoning). Mix levels — don't run all Easy or all Hard.
- You are not expected to ask every question. Pick 4–6 questions per section based on the candidate's stated strengths and the role being simulated.
- Use the 'Trainer Notes' column as a quick answer-quality check, not a script to read aloud.
- Where possible, ask for a real example from the candidate's own project work rather than a textbook definition — this quickly separates memorized answers from understanding.
- Use the Evaluation Rubric on the last page to record scores and feedback immediately after the round while it's fresh.

### 1. Core Java & OOP Concepts

| # | Interview Question                                                                                        | Level  | Trainer Notes / Expected Answer Pointers                                                                      |
|---|-----------------------------------------------------------------------------------------------------------|--------|---------------------------------------------------------------------------------------------------------------|
| 1 | Explain the four pillars of OOP with a real example from a project you've built.                          | Easy   | Encapsulation, Inheritance, Polymorphism, Abstraction. Look for a concrete example, not textbook definitions. |
| 2 | What is the difference between method overloading and method overriding?                                  | Easy   | Compile-time vs runtime; same class vs subclass; return type/signature rules.                                 |
| 3 | Why is Java called 'platform independent'? What role does the JVM play?                                   | Easy   | Bytecode + JVM abstraction. Watch for confusing JDK/JRE/JVM.                                                  |
| 4 | What is the difference between == and .equals() for objects and Strings?                                  | Easy   | Reference vs value comparison; String pool behaviour with new String().                                       |
| 5 | Explain the difference between abstract classes and interfaces. When would you choose one over the other? | Medium | Multiple inheritance, default/static methods in interfaces (Java 8+), state vs contract.                      |

| #  | Interview Question                                                                                                 | Level  | Trainer Notes / Expected Answer Pointers                                                                                |
|----|--------------------------------------------------------------------------------------------------------------------|--------|-------------------------------------------------------------------------------------------------------------------------|
| 6  | What is immutability? How would you design an immutable class?                                                     | Medium | final fields, no setters, defensive copies for mutable fields, no subclassing.                                          |
| 7  | Explain how the Garbage Collector works at a high level. What can cause a memory leak in Java despite GC?          | Medium | Generational GC, mark-and-sweep basics; leaks via static references, unclosed resources, listeners.                     |
| 8  | What is the difference between StringBuilder, StringBuffer, and String? When would you use each?                   | Medium | Immutability, thread safety, performance in loops.                                                                      |
| 9  | Explain multithreading basics: how do you create a thread, and what is the difference between Runnable and Thread? | Medium | Extending Thread vs implementing Runnable; preferred practice of implementing Runnable/using ExecutorService.           |
| 10 | What are static and instance initializer blocks, and when does each run?                                           | Hard   | Order of execution: static blocks once on class load, instance blocks on every object creation before constructor body. |

*Suggested time: 20 minutes if covering all questions above.*

## 2. Collections Framework & Java 8 Features

| # | Interview Question                                                                                                                        | Level  | Trainer Notes / Expected Answer Pointers                                                                          |
|---|-------------------------------------------------------------------------------------------------------------------------------------------|--------|-------------------------------------------------------------------------------------------------------------------|
| 1 | Compare ArrayList and LinkedList in terms of performance and use cases.                                                                   | Easy   | Random access vs insertion/deletion cost; internal array vs doubly linked list.                                   |
| 2 | What is the difference between HashMap, LinkedHashMap, and TreeMap?                                                                       | Easy   | Ordering guarantees, performance, use of Comparable/Comparator in TreeMap.                                        |
| 3 | How does HashMap handle collisions internally?                                                                                            | Medium | hashCode/equals, bucket chaining, treeification into a red-black tree after a threshold (Java 8+).                |
| 4 | What is the difference between Comparable and Comparator?                                                                                 | Medium | compareTo() on the class itself vs external compare() logic; multiple sort orders.                                |
| 5 | Explain lambda expressions and functional interfaces with an example.                                                                     | Medium | @FunctionalInterface, single abstract method, syntax, common built-ins (Function, Predicate, Supplier, Consumer). |
| 6 | Walk through a Stream API pipeline that filters, transforms, and collects a list — e.g. get names of employees earning above a threshold. | Medium | filter/map/collect, Collectors.toList(), lazy evaluation of intermediate operations.                              |
| 7 | What are default and static methods in interfaces? Why were they introduced in Java 8?                                                    | Medium | Backward compatibility for evolving interfaces without breaking implementers.                                     |
| 8 | What is the Optional class and what problem does it solve?                                                                                | Hard   | Avoiding NullPointerException, explicit handling of absent values, misuse pitfalls (Optional as a field).         |

*Suggested time: 16 minutes if covering all questions above.*

## 3. Exception Handling

| # | Interview Question                                                                                 | Level  | Trainer Notes / Expected Answer Pointers                                                                                         |
|---|----------------------------------------------------------------------------------------------------|--------|----------------------------------------------------------------------------------------------------------------------------------|
| 1 | What is the difference between checked and unchecked exceptions? Give one example of each.         | Easy   | Compile-time enforced (IOException) vs runtime (NullPointerException, ArithmeticException).                                      |
| 2 | Explain try-with-resources. Why is it preferred over a manual finally block for closing resources? | Easy   | AutoCloseable, automatic resource closing, reduced boilerplate, avoids resource leaks.                                           |
| 3 | Can you have a try block without a catch block? Explain the possible combinations.                 | Medium | try-finally, try-with-resources without catch; at least one of catch/finally required.                                           |
| 4 | How would you design a custom exception hierarchy for a banking or food-delivery application?      | Medium | Base application exception, specific subclasses (InsufficientFundsException, OrderNotFoundException), when checked vs unchecked. |
| 5 | What is exception chaining and why is it useful?                                                   | Medium | Wrapping a lower-level exception as the cause of a higher-level one; preserves root cause for debugging.                         |
| 6 | In a Spring Boot REST API, how would you implement centralized exception handling?                 | Hard   | @ControllerAdvice / @ExceptionHandler, mapping exceptions to proper HTTP status codes and a consistent error response body.      |

*Suggested time: 12 minutes if covering all questions above.*

#### 4. Spring Core & Spring Boot

| # | Interview Question                                                                                             | Level  | Trainer Notes / Expected Answer Pointers                                                                         |
|---|----------------------------------------------------------------------------------------------------------------|--------|------------------------------------------------------------------------------------------------------------------|
| 1 | What is Dependency Injection and why does Spring use it? Name the types of DI Spring supports.                 | Easy   | Constructor, setter, field injection; loose coupling and testability; constructor injection preferred.           |
| 2 | What is the Spring IoC container and what is a Bean?                                                           | Easy   | ApplicationContext manages object lifecycle/configuration; a bean is an object managed by the container.         |
| 3 | What are the different Spring bean scopes? Explain singleton vs prototype.                                     | Medium | singleton (default, one instance per context), prototype, request, session; when each is appropriate.            |
| 4 | What is the difference between @Component, @Service, @Repository, and @Controller?                             | Medium | All are stereotypes of @Component; semantic distinction, @Repository adds exception translation.                 |
| 5 | Explain the Spring Bean lifecycle — from creation to destruction.                                              | Medium | Instantiation, dependency injection, @PostConstruct, usage, @PreDestroy, container shutdown.                     |
| 6 | What auto-configuration does Spring Boot provide, and how does @SpringBootApplication tie things together?     | Medium | @EnableAutoConfiguration, @ComponentScan, @Configuration bundled together; conditional beans based on classpath. |
| 7 | How would you externalize configuration (e.g. DB URL, ports) across dev/test/prod environments in Spring Boot? | Medium | application.yml/properties per profile, @Profile, @ConfigurationProperties, environment variables.               |

| #  | Interview Question                                                                              | Level | Trainer Notes / Expected Answer Pointers                                                                                |
|----|-------------------------------------------------------------------------------------------------|-------|-------------------------------------------------------------------------------------------------------------------------|
| 8  | What is Spring AOP and where have you used it, or where would it be useful?                     | Hard  | Cross-cutting concerns like logging, transactions, security; @Aspect, advice types (before/after/around).               |
| 9  | How does Spring Boot's embedded server (Tomcat) differ from a traditional WAR deployment model? | Hard  | Self-contained executable jar vs deploying to an external servlet container.                                            |
| 10 | What is circular dependency between beans, and how would you resolve it?                        | Hard  | Constructor injection is stricter (fails fast); setter injection or @Lazy can defer resolution; better fix is redesign. |

*Suggested time: 20 minutes if covering all questions above.*

## 5. REST API Design

| # | Interview Question                                                                                                              | Level  | Trainer Notes / Expected Answer Pointers                                                                              |
|---|---------------------------------------------------------------------------------------------------------------------------------|--------|-----------------------------------------------------------------------------------------------------------------------|
| 1 | Explain REST principles. What makes an API 'RESTful'?                                                                           | Easy   | Statelessness, resource-based URIs, HTTP verbs, uniform interface, representations.                                   |
| 2 | What is the difference between PUT and PATCH? Between POST and PUT?                                                             | Easy   | PUT replaces the full resource (idempotent), PATCH partially updates, POST creates (not idempotent).                  |
| 3 | How do you handle request validation in a Spring Boot REST controller?                                                          | Medium | @Valid / @Validated with Bean Validation annotations (@NotNull, @Size), binding errors handled via @ExceptionHandler. |
| 4 | How would you version a REST API, and what are the trade-offs of different approaches?                                          | Medium | URI versioning, header versioning, request-param versioning; trade-offs in discoverability vs cleanliness.            |
| 5 | Explain the difference between @RequestParam, @PathVariable, and @RequestBody.                                                  | Medium | Query params vs URI template vars vs request body deserialization.                                                    |
| 6 | How would you design pagination and sorting for a GET endpoint returning a large list?                                          | Medium | Pageable/Page in Spring Data, page/size/sort query params, response metadata (totalElements, totalPages).             |
| 7 | What HTTP status codes would you return for: resource created, resource not found, validation failure, and unauthorized access? | Medium | 201, 404, 400, 401/403 — check they know the distinction between 401 and 403.                                         |
| 8 | How do you document and test a REST API before handing it to a frontend team?                                                   | Hard   | Swagger/OpenAPI, Postman collections, contract-first vs code-first documentation.                                     |

*Suggested time: 16 minutes if covering all questions above.*

## 6. Spring Data JPA / Hibernate

| # | Interview Question                                                                            | Level | Trainer Notes / Expected Answer Pointers                                            |
|---|-----------------------------------------------------------------------------------------------|-------|-------------------------------------------------------------------------------------|
| 1 | What is the difference between JPA and Hibernate?                                             | Easy  | JPA is a specification; Hibernate is the most common implementation/provider.       |
| 2 | Explain the difference between JpaRepository, CrudRepository, and PagingAndSortingRepository. | Easy  | Hierarchy — JpaRepository extends PagingAndSortingRepository extends CrudRepository |

| # | Interview Question                                                                                                     | Level  | Trainer Notes / Expected Answer Pointers                                                                                  |
|---|------------------------------------------------------------------------------------------------------------------------|--------|---------------------------------------------------------------------------------------------------------------------------|
|   |                                                                                                                        |        | CrudRepository, adding batch and flush operations.                                                                        |
| 3 | What is the N+1 select problem, and how would you fix it?                                                              | Medium | Lazy-loaded associations triggering extra queries in a loop; fix with JOIN FETCH, @EntityGraph, or batch fetching.        |
| 4 | Explain the difference between FetchType.LAZY and FetchType.EAGER, and when you'd choose each.                         | Medium | Lazy loads on access (default for collections), eager loads immediately (default for @ManyToOne); performance trade-offs. |
| 5 | How do @OneToMany, @ManyToOne, and @ManyToMany mappings work? Give an example from a project.                          | Medium | mappedBy, owning vs inverse side, join tables for many-to-many.                                                           |
| 6 | What is the persistence context / first-level cache in Hibernate?                                                      | Medium | Session-scoped cache of managed entities, dirty checking, avoids duplicate SQL within a transaction.                      |
| 7 | How does @Transactional work in Spring, and what happens if an exception is thrown inside a transactional method?      | Hard   | Proxy-based, rollback on unchecked exceptions by default, rollbackFor to include checked exceptions, propagation types.   |
| 8 | How would you handle database schema evolution across environments (e.g. adding a column without breaking production)? | Hard   | Flyway/Liquibase migrations vs hibernate.ddl-auto; why ddl-auto=update is risky in production.                            |

*Suggested time: 16 minutes if covering all questions above.*

## 7. Microservices Architecture

| # | Interview Question                                                                                                                            | Level  | Trainer Notes / Expected Answer Pointers                                                                                       |
|---|-----------------------------------------------------------------------------------------------------------------------------------------------|--------|--------------------------------------------------------------------------------------------------------------------------------|
| 1 | What is a microservices architecture, and what problems does it solve compared to a monolith?                                                 | Easy   | Independent deployability, scalability, technology diversity; trade-off of added operational complexity.                       |
| 2 | What is service discovery, and why is it needed? Have you used Eureka or a similar tool?                                                      | Medium | Dynamic registration/lookup of service instances instead of hardcoded URLs; Eureka server/client roles.                        |
| 3 | What is an API Gateway, and what responsibilities does it typically take on?                                                                  | Medium | Single entry point, routing, load balancing, auth, rate limiting — reduces client-side complexity.                             |
| 4 | How do microservices communicate with each other? Compare synchronous (Feign/REST) and asynchronous (messaging) approaches.                   | Medium | Feign clients for sync REST calls; Kafka/RabbitMQ for async/event-driven; trade-offs in coupling and resilience.               |
| 5 | What is the circuit breaker pattern, and why is it important in a microservices system?                                                       | Medium | Prevents cascading failures when a downstream service is unhealthy; Resilience4j/Hystrix concepts, fallback methods.           |
| 6 | How would you handle a transaction that spans multiple microservices, e.g. placing an order that debits an account and creates a loan record? | Hard   | No distributed 2PC in practice; Saga pattern (choreography or orchestration), eventual consistency, compensating transactions. |
| 7 | How do you handle configuration management across many microservices?                                                                         | Hard   | Spring Cloud Config Server, centralized property management, environment-specific overrides.                                   |

| # | Interview Question                                                                                     | Level | Trainer Notes / Expected Answer Pointers                                                                                      |
|---|--------------------------------------------------------------------------------------------------------|-------|-------------------------------------------------------------------------------------------------------------------------------|
| 8 | What challenges have you faced (or would expect) when debugging an issue that spans multiple services? | Hard  | Distributed tracing (Sleuth/Zipkin), correlation IDs, centralized logging (ELK) — look for awareness beyond 'check the logs'. |

*Suggested time: 16 minutes if covering all questions above.*

## 8. Design Patterns

| # | Interview Question                                                                                                                 | Level  | Trainer Notes / Expected Answer Pointers                                                                                                       |
|---|------------------------------------------------------------------------------------------------------------------------------------|--------|------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Explain the Singleton pattern. What problems can arise with it in a multi-threaded environment?                                    | Easy   | Single instance, global access point; race conditions in lazy initialization, fixes via synchronized/double-checked locking or enum singleton. |
| 2 | Explain the Factory Method pattern with an example. How is it different from Abstract Factory?                                     | Medium | Factory Method creates one product via subclassing; Abstract Factory creates families of related products.                                     |
| 3 | Where have you seen or used the Builder pattern? Why is it useful for objects with many optional fields?                           | Medium | Avoids telescoping constructors, improves readability, immutability support; Lombok @Builder as a real-world shortcut.                         |
| 4 | Explain the Strategy pattern and give a scenario where it would replace a large if-else/switch block.                              | Medium | Encapsulating interchangeable algorithms/behaviors behind a common interface, e.g. different discount or payment strategies.                   |
| 5 | What is the Observer pattern, and where is it used in Spring or Java itself?                                                       | Medium | Publisher-subscriber; Spring ApplicationEvent/Listener, Java's own Observable (legacy).                                                        |
| 6 | How does the Dependency Injection pattern relate to the broader principle of 'programming to an interface, not an implementation'? | Hard   | Decoupling, testability via mocking, open/closed principle connection.                                                                         |

*Suggested time: 12 minutes if covering all questions above.*

## 9. Security (Spring Security & Auth)

| # | Interview Question                                                                          | Level  | Trainer Notes / Expected Answer Pointers                                                                                 |
|---|---------------------------------------------------------------------------------------------|--------|--------------------------------------------------------------------------------------------------------------------------|
| 1 | What is the difference between authentication and authorization?                            | Easy   | Who you are vs what you're allowed to do.                                                                                |
| 2 | How does Spring Security's filter chain work at a high level?                               | Medium | Chain of servlet filters intercepting requests before reaching the controller; authentication and authorization filters. |
| 3 | What is JWT, and why is it commonly used for securing REST APIs?                            | Medium | Stateless, self-contained token with claims and signature; avoids server-side session storage.                           |
| 4 | What is the difference between session-based authentication and token-based authentication? | Medium | Server-stored session vs stateless client-held token; scalability implications.                                          |



| # | Interview Question                                                                                              | Level | Trainer Notes / Expected Answer Pointers                                                                                                  |
|---|-----------------------------------------------------------------------------------------------------------------|-------|-------------------------------------------------------------------------------------------------------------------------------------------|
| 5 | What is CSRF, and why is it usually disabled for stateless REST APIs but not for browser-session-based apps?    | Hard  | Cross-Site Request Forgery relies on cookies/sessions; token-based APIs without cookies are less exposed but not immune to other attacks. |
| 6 | How would you secure sensitive configuration like database passwords and API keys in a Spring Boot application? | Hard  | Environment variables, Spring Cloud Config with encryption, vaults (e.g. HashiCorp Vault), never hardcoding in application.yml.           |

*Suggested time: 12 minutes if covering all questions above.*

## 10. SQL & Database

| # | Interview Question                                                         | Level  | Trainer Notes / Expected Answer Pointers                                                                                 |
|---|----------------------------------------------------------------------------|--------|--------------------------------------------------------------------------------------------------------------------------|
| 1 | What is the difference between INNER JOIN, LEFT JOIN, and RIGHT JOIN?      | Easy   | Matched rows only vs all rows from one side with NULLs for unmatched.                                                    |
| 2 | What is normalization, and can you explain up to 3NF with a quick example? | Medium | Eliminating redundancy, functional dependency basics; don't expect textbook precision, look for practical understanding. |
| 3 | What is an index, and how does it affect read vs write performance?        | Medium | Speeds up reads/lookups, adds overhead on inserts/updates; when to avoid over-indexing.                                  |
| 4 | Write a query to find the second-highest salary in an Employee table.      | Medium | Subquery with LIMIT/OFFSET or MAX with NOT IN, or window functions (RANK/DENSE_RANK).                                    |
| 5 | What is the difference between a primary key and a unique key?             | Easy   | One PK per table (no NULLs) vs multiple unique keys allowed (may allow one NULL depending on DB).                        |
| 6 | How would you diagnose a slow-running query in production?                 | Hard   | EXPLAIN plan, missing indexes, N+1 queries from the app layer, connection pool exhaustion.                               |

*Suggested time: 12 minutes if covering all questions above.*

## 11. Cloud Deployment & DevOps Basics

| # | Interview Question                                                            | Level  | Trainer Notes / Expected Answer Pointers                                                                    |
|---|-------------------------------------------------------------------------------|--------|-------------------------------------------------------------------------------------------------------------|
| 1 | What is the difference between a Docker image and a Docker container?         | Easy   | Image is the immutable blueprint; container is a running instance of that image.                            |
| 2 | Why would you containerize a Spring Boot microservice before deploying it?    | Medium | Consistency across environments, isolation, easier scaling and orchestration.                               |
| 3 | What role does a CI/CD pipeline play in delivering a Spring Boot application? | Medium | Automated build/test/deploy stages, faster and more reliable releases, catching regressions early.          |
| 4 | At a high level, what does Kubernetes give you that plain Docker doesn't?     | Medium | Orchestration — scaling, self-healing, service discovery, rolling deployments across many containers/nodes. |

| # | Interview Question                                                                                            | Level | Trainer Notes / Expected Answer Pointers                                           |
|---|---------------------------------------------------------------------------------------------------------------|-------|------------------------------------------------------------------------------------|
| 5 | How would you approach externalizing environment-specific configuration for a cloud deployment (dev/QA/prod)? | Hard  | Spring profiles, ConfigMaps/Secrets in Kubernetes, or a centralized config server. |

*Suggested time: 10 minutes if covering all questions above.*

## 12. Behavioral / HR Round

| # | Interview Question                                                                               | Level  | Trainer Notes / Expected Answer Pointers                                                                |
|---|--------------------------------------------------------------------------------------------------|--------|---------------------------------------------------------------------------------------------------------|
| 1 | Walk me through a project where you had to make a design trade-off. What did you choose and why? | Medium | Looking for reasoning, not just recall — did they weigh alternatives, or just implement what was asked? |
| 2 | Describe a bug that was difficult to track down. How did you eventually find and fix it?         | Medium | Debugging process, tools used, root-cause thinking vs quick patching.                                   |
| 3 | How do you approach learning a new framework or technology under a deadline?                     | Easy   | Structured learning approach, use of official docs, willingness to ask for help.                        |
| 4 | Tell me about a time you disagreed with a teammate's technical approach. How was it resolved?    | Medium | Communication and collaboration skills, not just technical correctness.                                 |